

OCaml vs TypeScript vs Go

Сравнение систем типов: честно и по существу

OCaml

из ML-семейства, 1996

TypeScript

JavaScript + типы, 2012

Go

простота прежде всего,
2012

Что такое система типов

Система типов — набор правил которые компилятор использует чтобы поймать ошибки до запуска программы. Чем мощнее система типов, тем больше ошибок компилятор находит сам, не давая программе упасть в продакшне.

Ключевой вопрос не «есть ли типы» — они есть у всех трёх. Вопрос: что именно можно выразить через систему типов и насколько компилятор помогает а не мешает.

TypeScript: типы как аннотации

TypeScript — это JavaScript с добавленными типами. Ключевое слово: добавленными. Типы существуют только при компиляции — в рантайме их нет.

`typescript`

```
// Описываешь тип вручную
type Alert = {
  deviceId: string;
  severity: "critical" | "warning";
};

// Но компилятор тебе доверяет
const a = JSON.parse(data) as Alert; // <-- дыра: any внутри

// Структурная типизация: "утка"
function process(a: { deviceId: string }) { ... }
process({ deviceId: "A", extraField: 42 }); // OK — лишнее игнорируется
```

Сильные стороны:

- Discriminated unions: type Shape = Circle | Rect | Triangle
- Структурная типизация — гибко для JavaScript экосистемы
- Огромная экосистема npm, типы для большинства библиотек
- Постепенное внедрение в существующий JavaScript код

Слабые стороны:

- any отключает все проверки — легальный escape hatch
- JSON.parse возвращает any — дыра прямо в stdlib
- null и undefined текут даже со strict mode
- Вывод типов слабее — часто нужны явные аннотации
- Типы стираются при компиляции — рантайм ничего не знает о типах

Go: типы как контракты, намеренно простые

Go выбрал простоту осознанно. Система типов намеренно ограничена ради быстрой компиляции и лёгкого онбординга. Это не недостаток — это философия.

go

```
// Нет алгебраических типов
// Severity — строка без гарантий
type Alert struct {
    DeviceID string
    Severity string // "critical" или что угодно
}

// Интерфейсы: структурная типизация
type Stringer interface { String() string }
// Любой тип с методом String() — неявно Stringer

// Ошибки — значения. Можно проигнорировать:
val, _ := riskyOperation() // _ = молча игнорируем ошибку
```

Сильные стороны:

- Молниеносная компиляция — крупный проект за секунды
- Интерфейсы без явной декларации — гибкость без бойлерплейта
- Отличная стандартная библиотека для сетевых сервисов
- Простой онбординг — язык освоить за неделю

Слабые стороны:

- nil panic в рантайме — нет защиты на уровне типов
- Нет алгебраических типов данных
- Нет exhaustive matching — добавил case, компилятор не скажет
- Generics появились только в 1.18 и всё ещё ограничены
- Ошибки легко проигнорировать через _

OCaml: типы как математика

OCaml пришёл из ML-семейства (Meta Language, 1973). Здесь типы — не аннотации и не контракты. Это логические утверждения о программе которые компилятор доказывает.

1. Вывод типов — не надо писать типы вообще

```
ocaml

(* Нет ни одной аннотации типа *)
let add x y = x + y
(* Компилятор выводит: val add : int -> int -> int *)

(* Весь pipeline — без единой аннотации *)
let pipeline source =
  source
  |> with_watermarks ~latency:3000
  |> window (module Telemetry) (tumbling 30_000)
  |> aggregate Rules.compute
  |> flat_map (Rules.check Rules.fleet)
(* Если типы не совпадают → ошибка при компиляции, не в рантайме *)
```

2. Алгебраические типы — «одно из нескольких»

Это главное отличие. OCaml может выразить что значение — ровно одно из нескольких вариантов, и компилятор заставит обработать каждый:

```
ocaml

(* В Go/TypeScript нет прямого аналога *)
type event =
  | Data      of value * timestamp
  | Watermark of timestamp
  | Retract   of value * timestamp

(* Компилятор ЗАСТАВИТ обработать все три случая *)
match event with
| Data (v, t)    -> process v t
| Watermark t    -> advance_clock t
| Retract (v, t) -> undo v t
(* Забыл Retract → Warning: pattern-matching not exhaustive *)
```

3. Exhaustive pattern matching — компилятор требует полноты

```
ocaml
```

```

type severity = Critical | Warning | Info

let color = function
  | Critical -> "red"
  | Warning  -> "orange"
(* Забыл Info → Warning: this pattern-matching is not exhaustive *)

(* Добавил новый вариант в тип → компилятор покажет ВСЕ места
   где его не обработали. В Go/TypeScript такой гарантии нет. *)

```

4. Нет null — вообще

```

ocaml

(* В OCaml нет null и undefined.
   Вместо этого — явный тип option: *)
type 'a option = None | Some of 'a

let find_device id =
  match Hashtbl.find_opt table id with
  | None      -> (* обязан обработать — компилятор не даст забыть *)
  | Some info -> info.max_speed

(* В TypeScript: null течёт даже со strict mode
   В Go: nil вызывает panic в рантайме
   В OCaml: невозможно случайно разыменовать None *)

```

5. Параметрический полиморфизм без боксинга

```

ocaml

(* Одна функция работает для любого типа
   без потери типобезопасности и без any *)
val map : ('a -> 'b) -> 'a Stream.t -> 'b Stream.t

let speeds = map (fun t -> t.speed_kmh) telemetry_stream
(* speeds : float Stream.t — компилятор знает точный тип *)

(* В TypeScript: нужна явная аннотация или получишь any
   В Go: до 1.18 — interface{}, с 1.18 — generics, но ограниченные *)

```

6. Модульная система с сигнатурами

```

ocaml

```

```
(* Интерфейс модуля — это тип *)
module type STATE_BACKEND = sig
  type t
  val get  : t -> string -> bytes option
  val set  : t -> string -> bytes -> unit
end

(* Функтор: принимает модуль как аргумент *)
module Make (B : STATE_BACKEND) = struct
  (* код не знает какой именно backend *)
  (* замена Memory → RocksDB = одна строка *)
end

(* Мощнее интерфейсов Go и типов TypeScript:
  модуль может содержать типы, функции, другие модули *)
```

Честное сравнение

Свойство	OCaml	TypeScript	Go
Вывод типов	Полный (Hindley-Milner)	Частичный	Минимальный
Null safety	Нет null в принципе	Опционально, утекает	nil panic в рантайме
Exhaustive matching	Компилятор требует	Частично	Отсутствует
Алгебраические типы	Да, первоклассные	Имитация через union	Отсутствуют
Дженерики	Да, с 1973	Да, мощные	С 1.18, ограниченные
Ошибки	Result, типизированные	Исключения / any	Значения, легко игнорировать
Escape hatch	Obj.magic (явный, редкий)	any (повсеместный)	interface{} / unsafe
Простота освоения	Сложнее	Легко для JS-разработчиков	Намеренно прост
Экосистема	Маленькая	Огромная (npm)	Большая
Скорость компиляции	Быстро	Медленно (tsc)	Очень быстро

Где OCaml реально выигрывает

Когда важна корректность

Финансовые системы, компиляторы, proof systems. Jane Street — один из крупнейших трейдеров в мире — использует OCaml потому что система типов делает целый класс ошибок физически невозможными.

Когда меняется схема данных

Добавил поле в тип → компилятор показал все места где надо обновить код. В Go и TypeScript это легко пропустить.

Когда нужно рефакторить

«If it compiles, it probably works» — это реальная практика у опытных OCaml-разработчиков. В TypeScript скомпилировавшийся код может падать из-за `any` или `null` в рантайме.

Когда нужна алгебра предметной области

Система типов позволяет выразить бизнес-логику точно. Неправильное состояние становится нетипизируемым — его нельзя создать, не то что передать в функцию.

Пример из нашего проекта miniFlink:

```
ocaml

(* Тип event делает невозможным:
   - забыть обработать Retract
   - перепутать Data и Watermark
   - получить null вместо значения *)
type 'a event =
  | Data      of 'a * timestamp
  | Watermark of timestamp
  | Retract   of 'a * timestamp

(* Весь pipeline типобезопасен без единой аннотации типа *)
let pipeline source =
  source
  |> Event.with_watermarks ~latency:(seconds 3)
  |> Pipe.enrich (module Telemetry) ~from:devices ~merge:(...)
  |> Pipe.window (module Telemetry) (Pipe.tumbling (seconds 30))
  |> Pipe.aggregate Rules.compute
  |> Pipe.flat_map (Rules.check Rules.fleet)
  |> Pipe.dedup (module Alert) ~rule:(...) ~cooldown:(minutes 5)
```

Где Go и TypeScript выигрывают

- Молниеносная компиляция
- Простой онбординг
- Отличная stdlib для сетей
- Горутины и каналы
- Идеален для микросервисов
- Огромная экосистема npm
- Постепенное внедрение в JS
- Лучший tooling (VSCode)
- Фронтенд + бэкенд одним языком
- Большое сообщество

Вывод

OCaml превосходит оба языка по выразительности системы типов и гарантиям корректности. Но это не делает его автоматически лучшим выбором для любой задачи.

Система типов OCaml позволяет сделать неправильные состояния нетипизируемыми — программа которая компилируется с большой вероятностью корректна. В TypeScript и Go это гарантировать нельзя.

Язык	Выбери если...
OCaml	Нужна максимальная корректность. Компилятор — партнёр. Финансы, компиляторы, критически важные системы.
TypeScript	Уже есть JavaScript. Нужна экосистема npm. Команда знает JS. Фронтенд + бэкенд.
Go	Микросервисы, сетевые сервисы, быстрый онбординг. Простота и скорость компиляции важнее выразительности типов.